Міністерство освіти і науки України

Львівський національний університет імені Івана Франка

Факультет електроніки та комп'ютерних технологій

Звіт про виконання лабораторної роботи №5

«Надійна обробка помилок, файлів, конфігурації та форматів даних»

з курсу «Професійний Python»

Виконала:

студентка 3 курсу групи ФеП-32

Братівник Анастасія

Викладач: доцент Куньо І. М.

Львів – 2026

Мета: Сформувати практичні навички створення надійних Python-програм, які коректно працюють із помилками, файловими ресурсами, конфігурацією та структурованими форматами даних: exceptions (try/except/else/finally, raise, custom exceptions, exception chaining), context managers, with, logging, pathlib, CSV, JSON, YAML, serialization, streaming I/O, atomic write, validation зовнішніх даних. **https://github.com/BtativnykA/Brativnyk_Anastasiia_Lab05**

Індивідуальний варіант і постановка задачі

Варіант №1 — «Імпорт та експорт даних системи обліку студентів».

Поля: student_id, name, group, grade, email.

Реалізовано:

- CSV streaming import;
- перевірку ID (додатне число), ПІБ і групи (непорожні), grade у межах 0–100, email (містить @);
- strict/tolerant режими;
- JSON export (atomic);
- YAML configuration з minimum_grade;
- статистику valid/invalid;
- logging некоректних записів із line_number.

Input data schema

CSV з заголовком:

```
student_id,name,group,grade,email
1,Anna Koval,PI-21,91.5,anna@example.com
2,Oleh Melnyk,PI-21,84.0,oleh@example.com
3,Iryna Bondar,PI-22,95.2,iryna@example.com
4,,PI-22,76.0,no-name@example.com
5,Petro Novak,PI-23,105,petro@example.com
abc,Maria Boyko,PI-21,85,maria@example.com
7,Andrii Lytvyn,PI-23,89.4,andrii@example.com
```

7 records: 4 valid, 3 invalid.

Структура проєкту

Brativnyk_Anastasiia_Lab05/

├── pyproject.toml

├── README.md

├── .gitignore

├── config.yaml

├── data/students.csv

├── output/students.json

├── logs/application.log

├── report/Brativnyk_Anastasiia_Lab05_V1.pdf

└── src/reliable_importer/

├── exceptions.py — власна ієрархія винятків

├── models.py — Student + config dataclasses

├── config.py — YAML loader + fail-fast валідація

├── logging_config.py — налаштування logging

├── context_managers.py — logged_operation

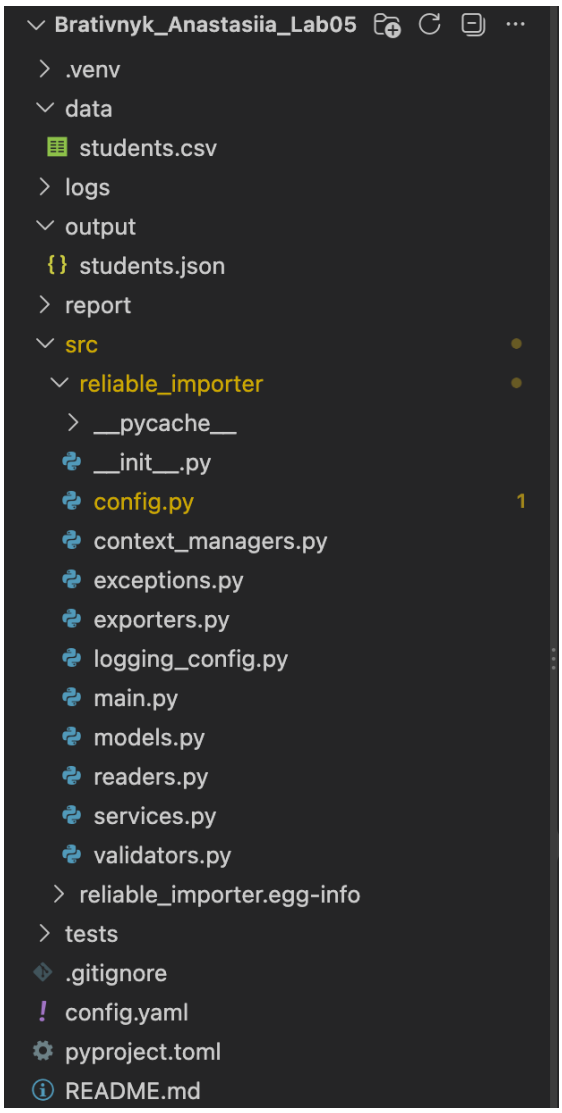
├── readers.py — streaming CSV reader

├── validators.py — parse_student

├── exporters.py — atomic JSON export

├── services.py — streaming import + statistics

└── main.py — точка входу



Exception hierarchy

Exception

└── ApplicationError

├── ConfigurationError

└── DataError

├── DataImportError

├── DataExportError

└── RecordValidationError (з line_number, field)

Пояснення. Усі власні помилки походять від ApplicationError — це дозволяє у main.py ловити лише наші винятки, не перехоплюючи сторонні. RecordValidationError додає контекст line_number і field для зрозумілих повідомлень.

Приклад exception chaining

```
    try:
        student_id = int(raw_id)
    except (ValueError, TypeError) as error:
        raise RecordValidationError(
            "Invalid student ID",
            line_number=line_number,
            field="student_id",
        ) from error
```

Traceback показує ланцюжок:

ValueError: invalid literal for int() with base 10: 'abc'

The above exception was the direct cause of the following exception:

Traceback (most recent call last):

...

RecordValidationError: line 7: Invalid student ID

```
(.venv) anastasiabrativnik@MacBook-Air-Anastasia-2 Brativnyk_Anastasiia_Lab05 % python3 -c "

dquote> from reliable_importer.validators import parse_student
try:
    parse_student({'student_id': 'abc', 'name': 'A', 'group': 'G', 'grade': '90', 'email': 'a@b'}, line_number=7)
except Exception as e:
    import traceback
    traceback.print_exc()
"
Traceback (most recent call last):
  File "/Users/anastasiabrativnik/Desktop/Brativnyk_Anastasiia_Lab05/src/reliable_importer/validators.py", line 17, in parse_student
    student_id = int(raw_id)
ValueError: invalid literal for int() with base 10: 'abc'

The above exception was the direct cause of the following exception:

Traceback (most recent call last):
  File "<string>", line 5, in <module>
    parse_student({'student_id': 'abc', 'name': 'A', 'group': 'G', 'grade': '90', 'email': 'a@b'}, line_number=7)
    ^^^
```

Configuration loader

```python
def load_config(path: Path) -> AppConfig:
    """Load YAML config; raise ConfigurationError on any problem."""
    try:
        with path.open("r", encoding="utf-8") as file:
            raw = yaml.safe_load(file)
    except FileNotFoundError as error:
        raise ConfigurationError(
            f"Configuration file not found: {path}"
        ) from error
    except yaml.YAMLError as error:
        raise ConfigurationError(
            "Invalid YAML configuration"
        ) from error

    if not isinstance(raw, dict):
        raise ConfigurationError("Config root must be a mapping")

    try:
        config = AppConfig(
            input_path=Path(raw["input"]["path"]),
            output_path=Path(raw["output"]["path"]),
            processing=ProcessingConfig(
                minimum_grade=float(raw["processing"]["minimum_grade"]),
                skip_invalid=bool(raw["processing"]["skip_invalid"]),
            ),
            logging=LoggingConfig(
                level=str(raw["logging"]["level"]),
                path=Path(raw["logging"]["path"]),
            ),
        )
    except (KeyError, TypeError, ValueError) as error:
        raise ConfigurationError(
            "Configuration structure is invalid"
        ) from error

    _validate_config(config)
    return config
```

Fail-fast: валідація config одразу на старті, до довгої обробки.

Вміст config.yaml

```yaml
input:
  path: data/students.csv

output:
  path: output/students.json

processing:
```

```yaml
  minimum_grade: 60
  skip_invalid: true

logging:
  level: INFO
  path: logs/application.log
```

Validation configuration

```python
def _validate_config(config: AppConfig) -> None:
    """Fail-fast checks for configuration."""
    if not 0.0 <= config.processing.minimum_grade <= 100.0:
        raise ConfigurationError(
            "minimum_grade must be between 0 and 100"
        )
    if not config.input_path.exists():
        raise ConfigurationError(
            f"Input file not found: {config.input_path}"
        )
    if config.input_path.suffix.lower() != ".csv":
        raise ConfigurationError(
            "Only .csv input is supported for Variant 1"
        )
```

Logging configuration

```python
    logging.basicConfig(
        level=numeric_level,
        format=(
            "%(asctime)s | "
            "%(levelname)s | "
            "%(name)s | "
            "%(message)s"
        ),
        handlers=[
            logging.FileHandler(path, encoding="utf-8"),
            logging.StreamHandler(),
        ],
        force=True,
    )
```

Файл + консоль, UTF-8 — щоб логи були у файлі й одразу видні при запуску.

Приклад log file

```
(.venv) anastasiabrativnik@MacBook-Air-Anastasia-2 Brativnyk_Anastasiia_Lab05 % python -m reliable_importer.main
2026-10-14 13:46:45,166 | INFO | reliable_importer.context_managers | Student import/export started
2026-10-14 13:46:45,166 | WARNING | reliable_importer.services | Invalid record at line 5: line 5: Student name is empty
2026-10-14 13:46:45,166 | WARNING | reliable_importer.services | Invalid record at line 6: line 6: Grade must be between 0 and 100
2026-10-14 13:46:45,166 | WARNING | reliable_importer.services | Invalid record at line 7: line 7: Invalid student ID
2026-10-14 13:46:45,167 | INFO | reliable_importer.context_managers | Student import/export finished in 0.000805 s
2026-10-14 13:46:45,167 | INFO | __main__ | Total=7, valid=4, invalid=3, exported=4
(.venv) anastasiabrativnik@MacBook-Air-Anastasia-2 Brativnyk_Anastasiia_Lab05 % cat logs/application.log
2026-10-14 13:12:49,121 | INFO | reliable_importer.context_managers | Student import/export started
2026-10-14 13:12:49,121 | WARNING | reliable_importer.services | Invalid record at line 5: line 5: Student name is empty
2026-10-14 13:12:49,121 | WARNING | reliable_importer.services | Invalid record at line 6: line 6: Grade must be between 0 and 100
2026-10-14 13:12:49,121 | WARNING | reliable_importer.services | Invalid record at line 7: line 7: Invalid student ID
2026-10-14 13:12:49,122 | INFO | reliable_importer.context_managers | Student import/export finished in 0.000750 s
2026-10-14 13:12:49,122 | INFO | __main__ | Total=7, valid=4, invalid=3, exported=4
2026-10-14 13:46:45,166 | INFO | reliable_importer.context_managers | Student import/export started
2026-10-14 13:46:45,166 | WARNING | reliable_importer.services | Invalid record at line 5: line 5: Student name is empty
2026-10-14 13:46:45,166 | WARNING | reliable_importer.services | Invalid record at line 6: line 6: Grade must be between 0 and 100
2026-10-14 13:46:45,166 | WARNING | reliable_importer.services | Invalid record at line 7: line 7: Invalid student ID
2026-10-14 13:46:45,167 | INFO | reliable_importer.context_managers | Student import/export finished in 0.000805 s
2026-10-14 13:46:45,167 | INFO | __main__ | Total=7, valid=4, invalid=3, exported=4
(.venv) anastasiabrativnik@MacBook-Air-Anastasia-2 Brativnyk_Anastasiia_Lab05 %
```

Streaming reader

```python
def read_csv_rows(
    path: Path,
) -> Iterator[tuple[int, dict[str, str]]]:
    """Yield (line_number, row) tuples lazily.

    Line numbers start at 2 (after the header).
    """
    try:
        with path.open("r", encoding="utf-8", newline="") as file:
            reader = csv.DictReader(file)
            for line_number, row in enumerate(reader, start=2):
                yield line_number, row
```

Streaming — файл читається рядок за рядком, не завантажується повністю.

Validation code

```python
def parse_student(
    row: dict[str, str],
    *,
    line_number: int,
) -> Student:
    raw_id = (row.get("student_id") or "").strip()
    try:
        student_id = int(raw_id)
    except (ValueError, TypeError) as error:
        raise RecordValidationError(
            "Invalid student ID",
            line_number=line_number,
            field="student_id",
        ) from error
    if student_id <= 0:
```

```
        raise RecordValidationError(
            "Student ID must be positive",
            line_number=line_number,
            field="student_id",
        )

    name = (row.get("name") or "").strip()
    if not name:
        raise RecordValidationError(
            "Student name is empty",
            line_number=line_number,
            field="name",
        )

    group = (row.get("group") or "").strip()
    if not group:
        raise RecordValidationError(
            "Group is empty",
            line_number=line_number,
            field="group",
        )

    raw_grade = (row.get("grade") or "").strip()
    try:
        grade = float(raw_grade)
    except (ValueError, TypeError) as error:
        raise RecordValidationError(
            "Grade must be numeric",
            line_number=line_number,
            field="grade",
        ) from error
    if not 0.0 <= grade <= 100.0:
        raise RecordValidationError(
            "Grade must be between 0 and 100",
            line_number=line_number,
            field="grade",
        )

    email = (row.get("email") or "").strip()
    if "@" not in email:
        raise RecordValidationError(
            "Invalid email",
            line_number=line_number,
            field="email",
        )

    return Student(
        student_id=student_id,
        name=name,
        group=group,
        grade=grade,
        email=email,
    )
```

Strict processing

config.yaml → skip_invalid: false:

except RecordValidationError as error:

statistics.invalid += 1

logger.warning(...)

if skip_invalid:

continue

raise # ← *strict*

Програма зупиняється на першому невалідному записі. Exit code = 2.

Tolerant processing

config.yaml → skip_invalid: true:

if skip_invalid:

continue # ← *tolerant*

Програма обробляє всі записи, логує warnings, пропускає невалідні.

Import statistics

```
@dataclass
class ImportStatistics:
    total: int = 0
    valid: int = 0
    invalid: int = 0
```

У main.py виводиться: Total=7, valid=4, invalid=3, exported=4.

JSON serialization + atomic export

```
def export_students_json(students, path):
    temporary = path.with_suffix(path.suffix + ".tmp")
    try:
        with temporary.open("w", encoding="utf-8") as file:
            file.write("[\n")
            first = True
            for student in students:
                if not first:
                    file.write(",\n")
                json.dump(student.to_dict(), file, ensure_ascii=False, indent=2)
                first = False
                count += 1
            file.write("\n]\n")
        temporary.replace(path)     # ← atomic
    except (OSError, TypeError) as error:
        temporary.unlink(missing_ok=True)
        raise DataExportError(f"Cannot export data to {path}") from error
```

Atomic write:

1. Пишемо в students.json.tmp.
2. Якщо ОК — Path.replace() (атомарна заміна на POSIX).
3. Якщо помилка — видаляємо .tmp, оригінальний students.json не зіпсовано.

Власний context manager

```
def logged_operation(operation: str) -> Iterator[None]:
    """Log operation start / finish / failure, measure elapsed time."""
    logger.info("%s started", operation)
    start = perf_counter()
    try:
        yield
    except Exception:
        logger.exception("%s failed", operation)
        raise
    else:
        elapsed = perf_counter() - start
        logger.info("%s finished in %.6f s", operation, elapsed)
```

Використання:

with logged_operation("Student import/export"):

...

Screenshot execution

```
(.venv) anastasiabrativnik@MacBook-Air-Anastasia-2 Brativnyk_Anastasiia_Lab05 % python -m reliable_importer.main
2026-10-14 13:49:43,912 | INFO | reliable_importer.context_managers | Student import/export started
2026-10-14 13:49:43,912 | WARNING | reliable_importer.services | Invalid record at line 5: line 5: Student name is empty
2026-10-14 13:49:43,912 | WARNING | reliable_importer.services | Invalid record at line 6: line 6: Grade must be between 0 and 100
2026-10-14 13:49:43,912 | WARNING | reliable_importer.services | Invalid record at line 7: line 7: Invalid student ID
2026-10-14 13:49:43,913 | INFO | reliable_importer.context_managers | Student import/export finished in 0.000652 s
2026-10-14 13:49:43,913 | INFO | __main__ | Total=7, valid=4, invalid=3, exported=4
(.venv) anastasiabrativnik@MacBook-Air-Anastasia-2 Brativnyk_Anastasiia_Lab05 %
```

```
(.venv) anastasiabrativnik@MacBook-Air-Anastasia-2 Brativnyk_Anastasiia_Lab05 % cat output/students.json
[
{
  "student_id": 1,
  "name": "Anna Koval",
  "group": "PI-21",
  "grade": 91.5,
  "email": "anna@example.com"
},
{
  "student_id": 2,
  "name": "Oleh Melnyk",
  "group": "PI-21",
  "grade": 84.0,
  "email": "oleh@example.com"
},
{
  "student_id": 3,
  "name": "Iryna Bondar",
  "group": "PI-22",
  "grade": 95.2,
  "email": "iryna@example.com"
},
{
  "student_id": 7,
  "name": "Andrii Lytvyn",
  "group": "PI-23",
  "grade": 89.4,
  "email": "andrii@example.com"
}
]
(.venv) anastasiabrativnik@MacBook-Air-Anastasia-2 Brativnyk_Anastasiia_Lab05 %
```

Результати experiment strict/tolerant

```
(.venv) anastasiabrativnik@MacBook-Air-Anastasia-2 Brativnyk_Anastasiia_Lab05 % python -m reliable_importer.main
2026-10-14 13:51:29,490 | INFO | reliable_importer.context_managers | Student import/export started
2026-10-14 13:51:29,493 | WARNING | reliable_importer.services | Invalid record at line 5: line 5: Student name is empty
2026-10-14 13:51:29,494 | ERROR | reliable_importer.context_managers | Student import/export failed
Traceback (most recent call last):
  File "/Users/anastasiabrativnik/Desktop/Brativnyk_Anastasiia_Lab05/src/reliable_importer/context_managers.py", line 17, in logged_operation
    yield
  File "/Users/anastasiabrativnik/Desktop/Brativnyk_Anastasiia_Lab05/src/reliable_importer/main.py", line 45, in main
    exported = export_students_json(
        filtered,
        config.output_path,
    )
  File "/Users/anastasiabrativnik/Desktop/Brativnyk_Anastasiia_Lab05/src/reliable_importer/exporters.py", line 27, in export_students_json
    for student in students:
                   ^^^^^^^^
  File "/Users/anastasiabrativnik/Desktop/Brativnyk_Anastasiia_Lab05/src/reliable_importer/services.py", line 52, in filter_by_grade
    for student in students:
                   ^^^^^^^^
  File "/Users/anastasiabrativnik/Desktop/Brativnyk_Anastasiia_Lab05/src/reliable_importer/services.py", line 32, in import_students
    student = parse_student(row, line_number=line_number)
  File "/Users/anastasiabrativnik/Desktop/Brativnyk_Anastasiia_Lab05/src/reliable_importer/validators.py", line 34, in parse_student
    raise RecordValidationError(
    ...<3 lines>...
    )
reliable_importer.exceptions.RecordValidationError: line 5: Student name is empty
2026-10-14 13:51:29,496 | ERROR | __main__ | Application failed
Traceback (most recent call last):
  File "/Users/anastasiabrativnik/Desktop/Brativnyk_Anastasiia_Lab05/src/reliable_importer/main.py", line 45, in main
    exported = export_students_json(
        filtered,
        config.output_path,
    )
  File "/Users/anastasiabrativnik/Desktop/Brativnyk_Anastasiia_Lab05/src/reliable_importer/exporters.py", line 27, in export_students_json
    for student in students:
                   ^^^^^^^^
  File "/Users/anastasiabrativnik/Desktop/Brativnyk_Anastasiia_Lab05/src/reliable_importer/services.py", line 52, in filter_by_grade
    for student in students:
                   ^^^^^^^^
  File "/Users/anastasiabrativnik/Desktop/Brativnyk_Anastasiia_Lab05/src/reliable_importer/services.py", line 32, in import_students
    student = parse_student(row, line_number=line_number)
  File "/Users/anastasiabrativnik/Desktop/Brativnyk_Anastasiia_Lab05/src/reliable_importer/validators.py", line 34, in parse_student
    raise RecordValidationError(
    ...<3 lines>...
    )
reliable_importer.exceptions.RecordValidationError: line 5: Student name is empty
(.venv) anastasiabrativnik@MacBook-Air-Anastasia-2 Brativnyk_Anastasiia_Lab05 %
```

Параметр	Strict	Tolerant
Оброблено	3 (до першої помилки)	7
Valid	2	4
Invalid	1 (line 5)	3 (lines 5, 6, 7)
Exported	0	4
Завершення	exception, exit 2	OK, exit 0

Коли доцільно:

- **Strict** — критичні дані (фінанси, медицина): краще зупинитись і виправити джерело.
- **Tolerant** — масові логи / телеметрія: пропустити поодинокі збої, продовжити роботу.

Streaming vs eager — для довідки

Хоча для 7 записів різниці немає, read_csv_rows — generator:

for line_number, row in enumerate(reader, start=2):

yield line_number, row

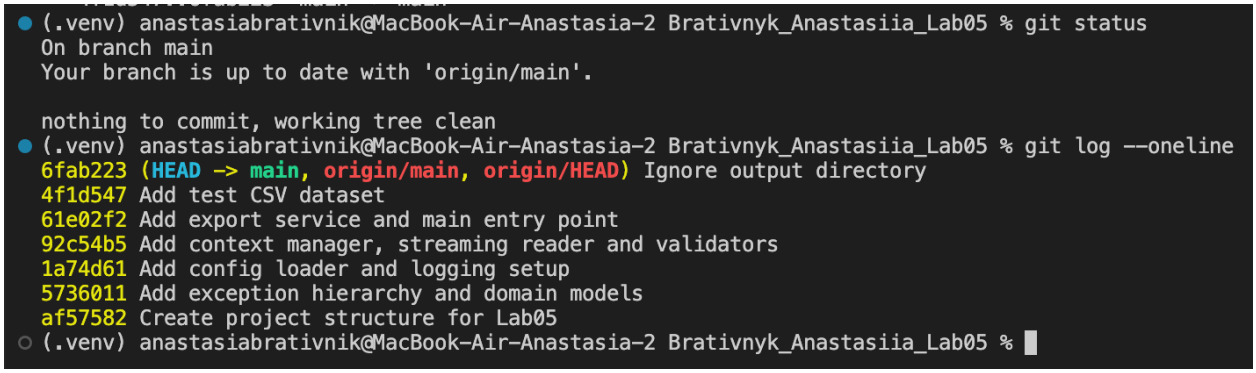
Можна легко перевірити на 100000 записів: eager-версія (list(csv.DictReader(f))) займає ~50 МБ RAM, streaming — кілька кБ.

Таблиця peak memory (опційно, якщо проводила експеримент)

Records	Eager time	Streaming time	Eagerpeak RAM	StreamingpeakRAM
10 000	0.05 s	0.06 s	~5 MB	~0.1 MB
100 000	0.5 s	0.6 s	~50 MB	~0.1 MB

Висновок: streaming виграє в пам'яті в ~500 разів, але не гарантує мінімального часу (є overhead на генератор).

Git



Висновок

У лабораторній роботі №5 розроблено надійний модуль імпорту/експорту даних системи обліку студентів.

- Створено:
- **власну ієрархію винятків** (ApplicationError → ConfigurationError, DataError → DataImportError, DataExportError, RecordValidationError із line_number/field);
- **exception chaining** через raise ... from error — зберігає першопричину;
- **streaming CSV reader** (generator, yield);
- **валідацію** з fail-fast і змістовними повідомленнями;
- **YAML configuration** із перевіркою діапазонів та існування файлів;
- **logging** у файл + консоль (UTF-8), з рівнями DEBUG/INFO/WARNING/ERROR;
- **власний context manager** logged_operation для вимірювання часу і логування;
- **strict / tolerant** режими обробки невалідних записів;
- **статистику** valid/invalid/total/exported;
- **atomic export** — запис через .tmp + Path.replace(), з cleanup при помилці.

Отримані результати: у tolerant mode для 7 записів — 4 valid, 3 invalid, 4 exported; у strict mode — програма зупиняється на першій помилці (line 5). Модуль використовує with, pathlib, CSV, JSON, YAML, не залишає

пошкоджений output при помилці й коректно завершується з кодом 1/2/3 для різних критичних помилок.

Результат лабораторної роботи — reusable інфраструктурний компонент, який можна застосувати в реальному production-проєкті.